Introduction to Lean 4

Kathmandu University Mathematics Students' Club (KUMSC)

So, What Are We Actually Doing?

The basic idea

In ordinary mathematics, a proof is a convincing explanation that a statement must be true. In Lean, we write that explanation precisely enough for a computer to check every step.

During this workshop, we will:

- learn about Lean, Mathlib, and open-source collaboration; and
- learn to translate simple statements from natural language into Lean's formal language.

What's a Proof?

Claim

If n and m are even natural numbers, then $n + m$ is even.

Proof, narrated

Suppose n and m are even. (the assumptions) By definition, $n = 2k$ and $m = 2\ell$ for some natural numbers k and ℓ. (unpacking the assumptions) Then $n + m = 2k + 2\ell = 2(k + \ell)$. (the computation) Since $k + \ell$ is a natural number, $n + m$ is twice a natural number and hence even, witnessed by $k + \ell$. (the conclusion)

A Proof That an Algorithm Is Correct

Algorithm: add one

Input: a natural number n.
Return $n + 1$.

Specification: what must the answer satisfy?

The returned number r must be a natural number greater than the input: $r > n$.

Proof

Let n be any natural number. The algorithm returns $r = n + 1$, which is also a natural number. Since $1 > 0$, we have $n + 1 > n$, so $r > n$. The algorithm returns immediately and satisfies the specification for every input. □

Example: input 3 gives output 4. The proof explains why this works for every natural number, not just 3.

What is a Proof Assistant?

Definition

A proof assistant is software that lets you state a mathematical claim and construct a proof of it that is mechanically checked, step by step, by a small trusted *kernel*.

A paper proof

- Checked by a human referee
- Gaps are possible (“clearly…”, “it is easy to see…”)
- Trust scales badly with length

A Lean proof

- Checked by a program
- No gaps allowed — it compiles or it doesn’t
- Trust reduces to trusting a tiny kernel

What Is Lean?

- **Lean 4** is both a programming language and an interactive theorem prover.
- You write a precise mathematical statement, then construct a proof while Lean continuously shows what remains to be proved.
- Lean's small trusted kernel checks the finished proof; tactics help build it, but they cannot make an invalid proof pass.
- Its community library, **Mathlib**, provides reusable definitions and theorems from many areas of mathematics.

The key experience today

Lean turns proof-writing into a conversation: you make one justified move, inspect the new goal, and continue until no goals remain.

Why Proof Assistants Matter

Theorem proving

- Every logical step is checked by a small trusted kernel.
- Hidden assumptions and missing cases must be made explicit.
- Large proofs can be organized into reusable, searchable libraries.

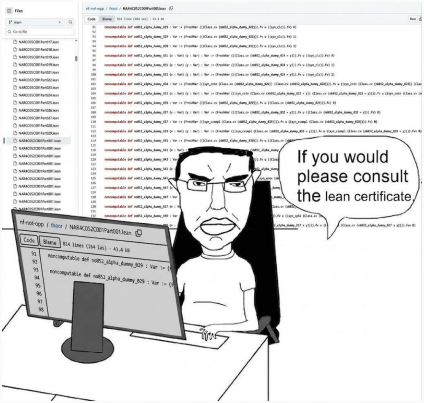
Software correctness

- A specification states precisely what a program should do.
- A proof can show that the implementation satisfies that specification.
- This is especially valuable for compilers, protocols, security tools, and other high-assurance systems.

The shared idea

Whether checking a theorem or a program, a proof assistant replaces “it seems correct” with a precise claim and a machine-checked justification.

When someone asks if their code works?



Lean Is Open and Collaborative

- Lean is open-source software: anyone can use it, inspect how it works, report problems, and contribute improvements.
- Mathlib is a shared library built by a worldwide community. Definitions, proofs, documentation, and teaching material are developed openly and reviewed together.
- A contribution does not have to be a famous theorem. Clarifying documentation, formalizing an elementary result, improving code, and helping another learner all matter.
- Because proofs are checked by the same kernel, contributions can be evaluated by their correctness rather than by the contributor's institution, location, or status.

Mathematics is something everyone can participate in

You do not need to begin as an expert. Open tools and shared libraries let learners build on existing work, ask questions, and gradually contribute mathematics of their own.

What's a Type?

Plain-language definition

A type is a **category of thing** — much like "a number," "a piece of text," and "a mathematical statement" are different categories.

- `5` has type $\mathbb{N}$: it is a natural number.
- `"hello"` has type `String`: it is text.
- `2 + 2 = 4` has type `Prop`: it is a proposition that can be proved.
- Lean checks that every expression is used with the correct type; many beginner errors are simply mismatched categories.

What's a Function?

Plain-language definition

A function is a rule that accepts input and produces output. In Lean, $A \to B$ means a function taking a value of type A and returning a value of type B.

For multiple inputs, list the parameters inside the declaration:

```
def add (a b : ℕ) : ℕ := a + b
#check add  -- ℕ→ℕ→ℕ
```

A note on currying

Lean reads $\mathbb{N} \to \mathbb{N} \to \mathbb{N}$ as $\mathbb{N} \to (\mathbb{N} \to \mathbb{N})$: first give **a**, then receive a function waiting for **b**. This is called *currying* and differs from tuple-style arguments common in many languages.

Terms and Types: the Big Idea, Made Precise

Formally:

- A **term** is a specific piece of data — `5`, `"hello"`, `2 + 2 = 4` are all terms.
- Every term has exactly one **type**, written `term : Type` — e.g. `5 : ℕ`.
- **How Lean uses this:** before running or checking anything, Lean figures out the type of every term involved — almost every error message you'll see today is Lean telling you two types didn't match where it expected them to.

A First Taste

Before any theory, let's just run something:

```
#eval 1 + 1        -- computes: 2
#eval "Hello, Lean!"
```

- `#eval` just *runs* an expression, like a calculator/interpreter.
- Nothing here is a "proof" yet — but it shows Lean is a live, responsive environment, not a batch compiler you wait on.

Choosing an Environment

Web editor

`https://live.lean-lang.org`

- Zero install
- Works instantly on any laptop
- Best choice for a workshop

Local (VS Code + elan)

- Install `elan` (Lean version manager)
- Install the `lean4` VS Code extension
- Faster once set up, but first Mathlib build is heavy (large download/compile)

Why this choice matters: a workshop's biggest time-sink is almost always installation, not Lean itself — default to the web editor unless you already have a local setup.

The Infoview — your main tool

- As you place your cursor inside a proof, a side panel (the Infoview) shows the current **goal state**: what hypotheses you have, and what remains to be proved.
- Errors appear as red squiggles *exactly* at the point they occur — you rarely need to guess where something went wrong.
- **How to read it:** everything above the line ⊢ is what you're allowed to use; everything after ⊢ is what you still owe.

```
n : ℕ
h : n > 0⊢
 n ≠ 0
```

`#check` vs `#eval`

```
#check 1 + 1        -- 1 + 1 : ℕ        (reports the TYPE)
#eval  1 + 1        -- 2                (reports the VALUE)

#check fun n => n + 1
#eval  (fun n => n + 1) 5   -- 6
```

Why two commands? `#check` answers "what *category* of thing is this"; `#eval` answers "what does this *compute to*." Keep both open mentally — almost every beginner error is a type mismatch (wrong category), not a value mismatch (wrong number).

Declaring Values and Variables

```
def answer : ℕ := 42      -- define a value

variable (n : ℕ)          -- declare an arbitrary natural number
variable (P Q : Prop)      -- declare arbitrary propositions

#check n + 1               -- uses the declared variable
#eval let y := 3; y + 1    -- a temporary local name
```

- `def` gives a name to a specific reusable value.
- `variable` introduces an arbitrary placeholder of a stated type; it does not choose a value.
- `let` creates a temporary local name inside one expression.

Writing Your First Functions

Start with `import Mathlib`, then name a few values and functions:

```
import Mathlib

def double (n : ℕ) := n + n
def two : ℕ := 2
def square (n : ℕ) := n * n

#eval double 21        -- 42
#eval square 6          -- 36
#eval two              -- 2
```

No `return` keyword, no braces — the body *is* the value, computed directly from the parameters.

Propositions Are Types; Proofs Are Values

Lean uses the same type-checking idea for programs and for proofs:

Expression	Its type	Meaning
`5`	$\mathbb{N}$	a value that is a natural number
`"hello"`	`String`	a value that is text
`h`	`P`	a value that is a proof of proposition `P`

The propositions-as-types idea

Each proposition `P` is treated as a type. To prove `P`, we must construct a term (or value) whose type is `P`.

When Lean checks a theorem, it is checking that the proof we supplied really has the type stated by the theorem.

What Is Mathlib?

A shared mathematical library

Mathlib is the community-maintained mathematics library for Lean 4. It provides definitions and proved theorems that you can reuse in your own proofs.

- It covers areas such as algebra, analysis, topology, and number theory.
- Instead of proving every fact from scratch, you can build on results already checked by Lean.
- Writing `import Mathlib` at the top of a Lean file makes the library available.
- It is open source: anyone can propose contributions, which are tested and reviewed.

Connecting this to propositions as types

A library theorem is a named proof term. Reusing it supplies evidence for the proposition you want to prove.

Logic Starts with Statements

A proposition

A proposition is a statement that can be true or false. In Lean, propositions have type `Prop`.

- "$2 + 2 = 4$" is a proposition.
- "n is even" is a proposition whose meaning depends on n.
- `P` and `Q` are commonly used as placeholders for arbitrary propositions.
- Questions, commands, and definitions are not themselves propositions: they do not assert something that can be proved.

Logic gives us a small vocabulary for joining simple propositions into more complicated ones.

From Truth Tables to Proofs

The truth-table approach

For a statement such as $P \to P$, list the possible truth values of P and check that the whole statement is true in every row.

The proof-building approach

Assume we have a proof of P. To prove P again, return that same proof. Lean checks this construction rather than asking us to fill in a truth table.

- Truth tables are useful for finite combinations of propositional variables.
- Proof rules also let us reason about arbitrary numbers, quantifiers, and mathematical definitions.
- Today we will build proofs from assumptions, one justified step at a time.

The Logical Connectives in Lean

Symbol	Meaning	Lean name
$P \to Q$	implication	function type `P → Q`
$P \wedge Q$	and	`And`
$P \vee Q$	or	`Or`
$\neg P$	not	`Not` (defined as `P → False`)
$\forall x, P(x)$	for all	dependent function type
$\exists x, P(x)$	exists	`Exists`

Why `¬P` is `P → False`: "P is false" is formalized as "P implies an absurdity" — if you could prove `False` from `P`, then `P` itself cannot hold. This one definition is why `intro` works uniformly on negations too.

Quantifiers: “For Every” and “There Exists”

Quantifiers tell us which values a statement is about.

```
#check ∀( n : ℕ, n + 0 = n)
#check ∃( k : ℕ, 4 = 2 * k)
```

- **Universal (∀):** the claim must hold for every value in the stated type. Checking one example is not enough.
- **Existential (∃):** at least one suitable value must exist. Such a value is called a *witness*; here, $k = 2$ works.
- Before the comma: the variable and its type. After the comma: the statement about that variable.

`#check` confirms that these are statements, not that they are true.

Assumptions and Conclusions

Most proofs can be read as a movement from information we are allowed to use toward a statement we must establish.

Natural language	Logical shape	How to read it
If P, then Q	$P \to Q$	assume P; prove Q
P and Q	$P \wedge Q$	prove both parts
P or Q	$P \vee Q$	prove at least one side
For every x, $P(x)$	$\forall x, P(x)$	choose an arbitrary x
There exists x with $P(x)$	$\exists x, P(x)$	provide a witness x

The shape of the goal usually tells us the first proof step to take.

A Translation Method

Turn an English sentence into a Lean `Prop`:

1. **Find the quantifiers.** “Every” becomes ∀; “there is” becomes ∃.
2. **Find the connectives.** “And” becomes ∧; “or” becomes ∨; “if … then” becomes →.
3. **Name the variables and their types.** For numbers, we will use ℕ; for arbitrary statements, `Prop`.
4. **Check the scope.** Use parentheses to show which parts belong together, then read the result back in English.

We are writing the *statement* first; we will learn to prove it next.

Worked Translations

```
-- "If P and Q hold, then P holds."
#check ∀( P Q : Prop, P ∧ Q → P)

-- "If P or Q holds, then Q or P holds."
#check ∀( P Q : Prop, P ∨ Q → Q ∨ P)

-- "Every natural number equals itself."
#check ∀( n : ℕ, n = n)

-- "There is a natural number equal to 6."
#check ∃( n : ℕ, n = 6)
```

#check checks the type of each statement; it does not prove it.

Your Turn: Translate These

Write the statements only — no proofs yet.

```
-- 1. "If Q holds, then P or Q holds."

-- 2. "If P implies Q and Q implies R, then P implies R."

-- 3. "For every natural number, there is a larger one."
```

Read your answer back

Identify the assumptions, the conclusion, and the scope of each quantifier. Does your Lean statement say exactly what the English sentence says?

Live Demo: Double Two Is Four

Using the `double` function we have just defined:

```
example : double 2 = 4 := by
  rfl
```

- **Before the proof step:** the Infoview shows the goal `double 2 = 4`.
- **After `rfl`:** the Infoview reports No goals to be solved — the proof is complete.

Proof Complete!



No goals remain. Lean has checked our proof.

`rfl`: Equality by Computation

```
example : double 2 = 4 := by
  rfl

example : double 2 = square 2 := by
  rfl
```

- `double 2` unfolds to `2 + 2`; `square 2` unfolds to `2 * 2`.
- Both compute to `4`, so `rfl` closes the equality.
- `rfl` proves equality when the two sides are equal by definition or computation; it is not a general arithmetic solver.

by: Start a Tactic Proof

```
example (P : Prop) : P → P := by
  intro h
  exact h
```

- Before `:= by`: the proposition we must prove.
- After **by**: indented instructions that build a proof term.
- The Infoview shows our assumptions and the remaining goal after each step.

Here `intro h` assumes **P**, and `exact h` supplies the proof we already have. We will examine these moves next.

The Syntax of a Theorem

```
theorem theoremName (inputs : Types) : statement := by
  proof steps
```

- `theoremName` is the reusable name of the result.
- The parentheses declare the objects or assumptions used by the theorem.
- Everything after the colon is the statement Lean expects us to prove.
- `:= by` begins a step-by-step tactic proof.

For a temporary unnamed theorem, use the same shape with `example`:

```
example (n : ℕ) : n = n := by
  rfl
```

`exact`: Use a Proof You Already Have

```
example (P Q : Prop) (h1 : P) (h2 : Q) : P := by
  exact h1
```

- `P` and `Q` are arbitrary propositions; `h1 : P` and `h2 : Q` are our assumptions.
- The goal is `P`. We already have `h1`, a proof of exactly that proposition.
- `exact h1` supplies that proof and closes the goal. We do not need to use every assumption.

Remember: `by` starts a tactic proof; `exact` supplies a term of the required type.

`intro`: Assume the Premise

```
example (P Q : Prop) : P → Q → P := by
  intro hp hq
  exact hp
```

- `intro hp` introduces `hp : P`; the goal becomes `Q → P`.
- `intro hq` introduces `hq : Q`; the goal becomes `P`.
- `exact hp` closes that goal.

We prove the implication by reasoning under its assumptions, not by assigning truth values to `P` and `Q`.

`constructor`: Prove Both Parts

```
example (P Q : Prop) (h1 : P) (h2 : Q) : P ∧ Q := by
  constructor
  · exact h1
  · exact h2
```

- A proof of `P ∧ Q` needs a proof of `P` and a proof of `Q`.
- `constructor` creates these two goals.
- Each bullet `·` starts the proof of one goal; `exact` supplies the corresponding assumption.

apply: Use an Implication

```
example (P Q : Prop) (p : P) (h : P → Q) : Q := by
  apply h p
```

`h` turns a proof of `P` into a proof of `Q`; `h p` is therefore a proof of our goal. The same reasoning can be written one step at a time:

```
example (P Q : Prop) (p : P) (h : P → Q) : Q := by
  apply h
  exact p
```

`apply h` changes the goal from `Q` to `P`; then `exact p` closes it.

Useful Existing Proof Terms

These basic logical tools are built into Lean; they do not require Mathlib.

```
#check And.left   -- P ∧ Q → P
#check And.right  -- P ∧ Q → Q
#check Or.inl     -- P → P ∨ Q
#check Or.inr     -- Q → P ∨ Q
```

- Given `h : P ∧ Q`, `And.left h` proves `P` and `And.right h` proves `Q`.
- `Or.inl` builds a proof using the left side of an “or”; `Or.inr` uses the right side.

They are proof terms, not tactics: use them with `exact` or `apply`.

cases: Handle Both Possibilities

```
example (P Q : Prop) : P ∨ Q → Q ∨ P := by
  intro h
  cases h with
  | inl hp => exact Or.inr hp
  | inr hq => exact Or.inl hq
```

- `intro h` gives us a proof of `P ∨ Q`.
- `cases h` considers its two possible forms: a proof of `P`, or a proof of `Q`.
- If we have `hp : P`, put it on the right of `Q ∨ P`; if we have `hq : Q`, put it on the left.

Your Turn: Logic Proofs

Replace each `sorry`. Use `intro`, `exact`, `constructor`, and `apply`.

```
-- 1. Return the matching assumption.
example (P Q : Prop) : P → Q → Q := by
  sorry

-- 2. Build a conjunction in the opposite order.
example (P Q : Prop) : P → Q → Q ∧ P := by
  sorry

-- 3. Apply two implications in sequence.
example (P Q R : Prop) (h1 : P → Q) (h2 : Q → R) :
    P → R := by
  sorry
```

`sorry` marks an unfinished proof; remove it to complete an exercise.

Your Turn: Cases and Proof Terms

Use `cases`, `constructor`, `intro`, and `exact`, with `And.left`, `And.right`, `Or.inl`, and `Or.inr`.

```
-- 4. Extract Q, then choose the left side of the "or".
example (P Q : Prop) : P ∧ Q → Q ∨ P := by
  sorry

-- 5. Handle both possibilities.
example (P Q : Prop) : Q ∨ P → P ∨ Q := by
  sorry

-- 6. Pair each case with the proof of R.
example (P Q R : Prop) (hr : R) :
    P ∨ Q → (P ∧ R) ∨ (Q ∧ R) := by
  sorry
```

A Definition About Numbers: `isEven`

```
def isEven (n : ℕ) : Prop := ∃ k : ℕ, n = 2 * k
```

- `isEven` takes a natural number and returns a proposition.
- Read it as: "There is a natural number `k` such that `n = 2 * k`."
- A hypothesis `h : isEven n` is a proof that `n` satisfies this definition.

Next: prove evenness by choosing a witness, then unpack evenness assumptions.

`unfold` and `use`: Prove Six Is Even

```
theorem e6 : isEven 6 := by
  unfold isEven
  use 3
```

- `unfold isEven` exposes the goal `∃ k : ℕ, 6 = 2 * k`.
- `use 3` supplies the witness. The remaining equality is true by computation, and `use` closes it automatically.
- `use` is a Mathlib tactic; our basic logical proof terms did not require Mathlib.

An existential proof gives a witness *and* a proof that it works.

have: Name a Proof We Will Use

```
example (n m : ℕ) (h : isEven n ∧ isEven m) :
    isEven n := by
  have h1 : isEven n := And.left h
  exact h1
```

have records an intermediate proof in the local context. It does not introduce an unjustified assumption.

Our next claim

The sum of two even natural numbers is even. We will extract both evenness proofs, unpack their witnesses, and build a witness for the sum.

obtain: Unpack an Evenness Proof

```
example (n : ℕ) (h : isEven n) :∃
     k : ℕ, n = 2 * k := by
  unfold isEven at h
  obtain (k, )hk := h
  use k, hk
```

- `unfold isEven at h` expands the definition in a hypothesis.
- `obtain ⟨k, hk⟩ := h` names the witness `k` and its equation `hk : n = 2 * k`.
- `use k, hk` supplies both the witness and the proof it satisfies the goal.

`calc` and `rw`: Calculate with Equations

```
#check Nat.left_distrib
-- 2 * (k1 + k2) = 2 * k1 + 2 * k2

example (n m k1 k2 : ℕ)
    (hn : n = 2 * k1) (hm : m = 2 * k2) :
    n + m = 2 * (k1 + k2) := by
  calc
    n + m = 2 * k1 + 2 * k2 := by rw [hn, hm]
    _ = 2 * (k1 + k2) := by
      rw ←[ Nat.left_distrib 2 k1 k2]
```

- `calc` builds a chain of equalities; `_` repeats the previous right-hand side.
- `rw` substitutes using an equation; `←` uses it in reverse.
- `Nat.left_distrib` is the natural-number form of the distributive law, also available as `mul_add` in Mathlib.

Why the Sum of Two Even Numbers Is Even

Informal proof

Assume n and m are even. Then $n = 2k_1$ and $m = 2k_2$ for some natural numbers k_1 and k_2. Therefore,

$$n + m = 2k_1 + 2k_2 = 2(k_1 + k_2).$$

So $k_1 + k_2$ is a witness that $n + m$ is even.

- `intro` assumes the conjunction; `have` extracts each part.
- `obtain` names the two witnesses and equations.
- `calc` and `rw` prove the required equality.
- `use` supplies the new witness and its equality proof.

Putting It Together: The Sum of Even Numbers

```
theorem sumoeven (n m : ℕ) :
    isEven n ∧ isEven m → isEven (n + m) := by
  intro h
  have h1 : isEven n := And.left h
  unfold isEven at h1
  obtain ⟨k1, ⟩hn := h1
  have h2 : isEven m := And.right h
  unfold isEven at h2
  obtain ⟨k2, ⟩hm := h2
  have h3 : n + m = 2 * (k1 + k2) := by
    calc
      n + m = 2 * k1 + 2 * k2 := by rw [hn, hm]
      _ = 2 * (k1 + k2) := by
        rw ←[ Nat.left_distrib 2 k1 k2]
  unfold isEven
  use k1 + k2, h3
```

The last line supplies the witness `k1 + k2` and the equality `h3` we just proved.

`omega` and `decide`

```
example (a b : ℕ) (h : a < b) : a + 1 ≤ b := by omega
example : (101 : ℕ) > 100 := by decide
```

- `omega`: a decision procedure for *linear arithmetic* over $\mathbb{N}/\mathbb{Z}$ — give it hypotheses and a goal built from $+, -, <, \leq, =$ with no variable multiplied by another, and it either closes the goal or fails outright.
- `decide`: brute-force evaluates a *concrete*, decidable proposition to `true`/`false` and checks the answer is `true` — great for one specific numeral fact, useless for a statement about an arbitrary variable.

Both ship with Lean itself — no library needed, which is why we could use them before ever mentioning Mathlib.

simp: Simplification

```
example (n : ℕ) : n + 0 = n := by simp
example (P : Prop) (h : P ∧ True) : P := by simp at h; exact h
```

How it works: `simp` repeatedly rewrites the goal using a curated database of "obviously simplifying" equations (the *simp set*) until nothing more applies. **Why it's powerful but risky:** it can solve a goal in one line, but when it fails, the resulting state can be hard to read — use it once you roughly expect it to close the goal, not as a first blind attempt.

`rw`: Rewriting with an Equation

```
example (a b : ℕ) (h : a = b) : a + 1 = b + 1 := by
  rw [h]

example (a b : ℕ) (h : a = b) : b + 1 = a + 1 := by
  rw ←[ h]
```

How it works: `rw [h]` literally replaces every occurrence of the left side of `h` with its right side in the goal (`←` reverses the direction). Unlike `simp`, it does exactly the substitutions you list — nothing more, nothing automatic.

Which Tactic, When? (a quick decision guide)

- Goal is true by definition/computation $\Rightarrow$ `rfl`
- Goal follows from a general simplification rule $\Rightarrow$ `simp`
- You have a specific equation to substitute $\Rightarrow$ `rw`
- Goal is linear arithmetic over $\mathbb{N}/\mathbb{Z}$ $\Rightarrow$ `omega`
- Goal is one concrete decidable fact $\Rightarrow$ `decide`
- You don't know the exact lemma name $\Rightarrow$ `exact?` / `apply?`

Finding Things in Mathlib

```
example (a b : ℕ) : a + b = b + a := by exact?
-- suggests: exact Nat.add_comm a b
```

- `exact?` / `apply?`: ask Lean to search Mathlib for a term that closes the current goal.
- Loogle (`https://loogle.lean-lang.org`): search by *type signature/shape*, not just by guessing a name.
- Naming convention (why lemma names look the way they do): `add_comm`, `Nat.succ_pos`, `mul_assoc` — names describe the *statement structure* (`add` + `comm` = "addition is commutative"), so once you know the convention you can often *guess* a lemma's name correctly.

Let's Prove Something Everyone Already Believes

Chosen theorem: Euclid's Theorem

There are infinitely many prime numbers.

Why this theorem, specifically?

- Everyone in the room already knows and believes it — the point is bridging *familiar* math to *formal* math, not learning new mathematics.
- The classical proof (Euclid, ∼300 BC) is short, elegant, and constructive — perfect for a 10-minute walkthrough.
- It is a genuine theorem in Mathlib, so we see the whole pipeline: informal idea → formal statement → real, checked proof.

(An equally good alternative, if you'd rather: $\sqrt{2}$ *is irrational* — also classical, also in Mathlib as `irrational_sqrt_two`.)

Resources

- Web editor: `https://live.lean-lang.org`
- *Mathematics in Lean* (free textbook): `https://leanprover-community.github.io/mathematics_in_lean/`
- Mathlib docs: `https://leanprover-community.github.io/mathlib4_docs/`
- Loogle: `https://loogle.lean-lang.org`
- Community chat: `https://leanprover.zulipchat.com`

Small Introduction to How It Works

A familiar idea: type-checking

A compiler checks that the value assigned to a variable matches its declared type: a variable declared to hold a number must be given a value of that type.

- Lean uses the same idea for proofs: the proposition is the required type, and the proof is the term supplied as its value.
- For `h : P`, Lean checks that the supplied term `h` really has type `P`.
- Lean's kernel checks this match. A proof term of a different type cannot establish `P`.